

23CSE211 - DESIGN AND ANALYSIS OF ALGORITHMS

Attempts on solving Flow-Free - 3

Dynamic Programming, Backtracking

Submitted by:

Akilan S S	–	CB.SC.U4CSE24707
Govind Nair	–	CB.SC.U4CSE24720
M V Sai Kartik	–	CB.SC.U4CSE24744
Sriram Tatikonda	–	CB.SC.U4CSE24753

Submitted to:

Dr. Vidhya Balasubramanian

Wednesday 11th March, 2026

Contents

1	Introduction	2
2	Dynamic Programming - Bellman Ford + Bitmasking	2
2.1	Description	2
2.1.1	Bellman-Ford Path	2
2.1.2	Bitmaskng	2
2.2	Time Complexity Analysis	3
2.3	Pseudocodes	3
3	A* + Dynamic Programming with Bitmask Representation	6
3.1	Description	6
3.2	Dynamic Programming Formulation	6
3.3	Backtracking Model	7
3.4	Time Complexity Analysis	7
3.5	Pseudocode	8
4	Backtracking	9
4.1	Description	9
4.2	Time complexity Analysis	9
4.3	Feasibility checks	9
4.3.1	DFS feasibility check	9
4.3.2	Dead region check	9
4.4	Zig-Zag Variant	10
5	Analysis across Algorithms	11
6	Gallery	14
7	Conclusion and Future Directions	15
8	LLM Disclosure	16

1 Introduction

Flow-free is a grid based game, where in an n by n grid, there are k pairs of colours (or terminals), which has to be connected by drawing pipes on the grid. No two pipes must intersect, and they should fill the grid as much as possible. In this project, we attempt to solve the game Flow-Free with traditional algorithm techniques, presented till date in 23CSE211, in a collaborative manner through the following strategies. In this final phase of the lab evaluation, we shall demonstrate attempts using dynamic programming, and solve the game using backtracking.

1. Dynamic Programming - Bellman Ford
2. Dynamic Programming
3. Backtracking

2 Dynamic Programming - Bellman Ford + Bitmasking

2.1 Description

This algorithm follows the structure of the BFS + sorting algorithm from the first solving attempt and builds on it with a bitmasking algorithm that finds the optimal pair ordering in such a manner that the maximum number of terminal pairs have a valid path.

2.1.1 Bellman-Ford Path

1. **Subproblem:** $dp[k][r][c]$ = shortest path from the starting node (terminal) to $(row, column)$ using at most k edges
2. **Guess:** We find the neighbour u which we came from to reach (r, c) in step k
3. **Relation between subproblems:** $dp[k][r][c] = \min(dp[k-1][nx][ny] + 1)$ for each neighbour $u (nx, ny)$
4. **Tabulation:** $dp[k][r][c]$ is computed from $k-1$ and $dp[0][startrow][startcol]$ is initialised to 0

2.1.2 Bitmaskng

1. **Subproblem:** $dp[mask]$ = minimum total path length to solve pairs encoded by 1 in mask
2. **Guess:** We find which pair i was solved last to reach this mask and which pair to add when transitioning from mask to $(mask | (1 << i))$
3. **Relation between subproblems:** $dp[mask | (1 << i)] = \min(dp[mask] + \text{length of path for pair } i \text{ on board}[mask])$
4. **Tabulation:** $dp[0] = 0$ and $dp[mask]$ is filled by iterating mask from 0 to $2^k - 1$

2.2 Time Complexity Analysis

1. **Bellman-Ford path:** $\mathcal{O}(n^4)$ or $\mathcal{O}(V^2 + VE)$ where E is $4V$ and V is n^2
2. **Sorting + pathfinding:** $\mathcal{O}(kn^4)$ where k is the number of pairs
3. **Bitmasking + pathfinding:** $\mathcal{O}(k2^k n^4)$ since masking runs for 2^k times where each mask tries to solve k unsolved pairs

2.3 Pseudocodes

Algorithm 1 Bellman-Ford pathfinder

```
1: function BELLMANFORDPATH(board, start, goal)
2:   Build edge list
3:   for each cell  $(r, c)$  in grid do
4:     if cell hasPipe or isTerminal and  $(r, c)$  is not start or goal then
5:       continue
6:     end if
7:     for each neighbour  $(nr, nc)$  do
8:       if neighbour hasPipe or isTerminal and  $(nr, nc)$  is not start or goal then
9:         continue
10:      end if
11:      add edge  $(r, c) \rightarrow (nr, nc)$ 
12:    end for
13:  end for
14:   $dp[0..V][r][c] \leftarrow \infty$ 
15:   $dp[0][start] \leftarrow 0$ 
16:  for  $k = 1$  to  $V$  do
17:     $dp[k] \leftarrow$  copy of  $dp[k - 1]$ 
18:    for each edge  $(u \rightarrow v)$  do
19:      if  $dp[k - 1][u] + 1 < dp[k][v]$  then
20:         $dp[k][v] \leftarrow dp[k - 1][u] + 1$ 
21:         $parent[k][v] \leftarrow u$ 
22:      end if
23:    end for
24:    if no updates then
25:      break
26:    end if
27:  end for
28:  find smallest  $k$  where  $dp[k][goal] \neq \infty$ 
29:  reconstruct path by walking  $parent[k]$  backwards from goal to start
30:  return path
31: end function
```

Algorithm 2 Bellman-Ford + sorting algorithm

```
1: function ALGORITHM3(board)
2:   collect all terminal pairs
3:   sort pairs by Euclidean distance (shortest first)
4:   remove already solved pairs
5:   for each pair in sorted order do
6:      $path \leftarrow \text{BELLMANFORDPATH}(\text{board}, \text{pair.a}, \text{pair.b})$ 
7:     if  $path$  is not empty then
8:       return  $path$ 
9:     end if
10:  end for
11:  return empty
12: end function
```

Algorithm 3 Bellman-Ford + bitmasking algorithm

```
1: function ALGORITHM4(board)
2:   collect all terminal pairs
3:   remove already solved pairs
4:    $n \leftarrow$  number of unsolved pairs
5:   if  $n > 12$  then
6:     return ALGORITHM3(board)
7:   end if
8:    $dp[0..2^n] \leftarrow \infty$ 
9:    $dp[0] \leftarrow 0$ 
10:   $boardAt[0] \leftarrow$  current board
11:  for  $mask = 0$  to  $2^n - 1$  do
12:    if  $dp[mask] = \infty$  then
13:      continue
14:    end if
15:    for each unsolved pair  $i$  not in  $mask$  do
16:       $path \leftarrow$  BELLMANFORDPATH( $boardAt[mask]$ ,  $pair[i].a$ ,  $pair[i].b$ )
17:      if  $path$  is empty then
18:        continue
19:      end if
20:       $newMask \leftarrow mask \mid (1 \ll i)$ 
21:       $score \leftarrow dp[mask] + path.length$ 
22:      if  $score < dp[newMask]$  then
23:         $dp[newMask] \leftarrow score$ 
24:         $from[newMask] \leftarrow mask$ 
25:         $pathUsed[newMask] \leftarrow path$ 
26:         $boardAt[newMask] \leftarrow boardAt[mask]$  with  $path$  applied
27:      end if
28:    end for
29:  end for
30:   $bestMask \leftarrow$  mask with most bits set (ties broken by lowest  $dp$  score)
31:  trace back through  $from[]$  until  $from[cur] = 0$ 
32:  return  $pathUsed[cur]$  ▷ first path of optimal sequence
33: end function
```

3 A* + Dynamic Programming with Bitmask Representation

3.1 Description

The A* with DP Bitmask solver extends the existing `algorithm.cpp` by combining A*-based terminal pair connection with a bitmask-accelerated dynamic programming expansion phase. A* first automatically routes each color pair, producing an initial set of connected paths on the board. This intermediate board state is then converted into a compact bitmask representation, where each color occupies a 64-bit integer (`uint64_t`) encoding the positions of all its cells. A DP memoization table indexed by the full vector of color masks records the minimum cost to reach each configuration, enabling efficient backtracking and state reuse.

The algorithm operates in two tightly coupled phases. In the **connection phase**, A* iterates over all terminal pairs and fills in initial paths without human interaction. In the **expansion phase**, DP uses bitwise frontier detection—computing empty neighbors of each color in $\mathcal{O}(1)$ per cell via precomputed neighbor masks—to explore all valid expansions, memoize visited states, and backtrack when no progress is possible. Together, these phases reduce redundant computation and avoid revisiting equivalent board configurations, yielding a solver that is both systematic and efficient.

3.2 Dynamic Programming Formulation

1. **Subproblem:** Given a board configuration encoded as a vector of color masks, find the maximum number of empty cells that can still be filled:

$$dp[\{colorMask_1, \dots, colorMask_k\}] = \text{max cells fillable from this state}$$

2. **Guess:** At each step we guess which color c expands and into which adjacent empty cell x :

$$(c, x) \quad \text{where } x \in frontierCells(c)$$

3. **Subproblem Relation:** Expanding color c into cell x updates the state:

$$colorMask[c] \mid= (1 \ll x), \quad filledMask \mid= (1 \ll x)$$

giving the recurrence:

$$dp[state] = \max_{\text{valid}(c,x)} (1 + dp[next_state])$$

4. **Recursion:** `solve(state)` tries every valid (c, x) expansion recursively, undoing each move on return (backtracking).
5. **Memoization:** Solved states are stored in:

`unordered_map<State, int> dp`

Before entering recursion, if `state ∈ dp`, return immediately. This prevents recomputing equivalent board configurations reached via different expansion orderings.

3.3 Backtracking Model

The partial solution is modeled as a move sequence:

$$\text{vector}\langle \text{Move} \rangle \text{ currentPath}, \quad \text{Move} = \{\text{color}, \text{cellIndex}\}$$

For example:

$$[(R, 12), (R, 13), (B, 7), (G, 21)]$$

means red expands to cells 12 and 13, then blue to 7, then green to 21.

Completion condition. The solution is complete when every color has an empty frontier:

$$\text{frontierCells}(c) = 0 \quad \forall c$$

Feasibility check. A move (c, x) is valid iff cell x is empty and adjacent to color c , which `frontierCells()` already guarantees by construction—so no separate check is needed.

3.4 Time Complexity Analysis

Neighbor Precomputation: $\mathcal{O}(N^2)$ Each cell's neighbor mask is computed once at initialization by scanning the four cardinal directions. Total work across all N^2 cells is linear in the board size.

Frontier Detection per Color: $\mathcal{O}(d)$ Where d is the number of cells currently assigned to that color. Each cell is extracted from the mask using `_builtin_ctzll` and ANDed against the precomputed neighbor mask. The final frontier is masked against `filledMask` to retain only empty cells.

A* Path Search: $\mathcal{O}(N^2 \log N)$ For a grid of size $N \times N$:

$$V \leq N^2, \quad E = \mathcal{O}(N^2)$$

Using a binary heap priority queue:

$$\mathcal{O}(V \log V) = \mathcal{O}(N^2 \log N)$$

DP Expansion (single state): $\mathcal{O}(K \cdot d_{\max})$ At each DP state, frontier detection runs for each of K colors. The work per color is proportional to the number of cells it currently occupies, bounded by N^2 in the worst case.

Full Puzzle with Memoization: $\mathcal{O}(S \cdot K \cdot N^2)$ Where S is the number of distinct states explored before memoization cuts off repeated configurations. In practice $S \ll 2^{N^2}$ due to the bitmask hash table pruning equivalent states.

Summary

$$\text{A* connection phase} \rightarrow \mathcal{O}(K \cdot N^2 \log N)$$

$$\text{DP expansion phase (worst case)} \rightarrow \mathcal{O}(S \cdot K \cdot N^2)$$

3.5 Pseudocode

Algorithm 4 A* + DP Bitmask Solver

```

1: function INITIALIZE(board)
2:   Compute neighborMask[i] for all cells  $i \in [0, rows \times cols)$ 
3:   Run A* for each terminal pair to produce initial connected paths
4:   Call initializeMasks(board) to populate colorMask[] and filledMask
5: end function
6: function FRONTIERCELLS(color)
7:   frontier  $\leftarrow 0$ 
8:   cells  $\leftarrow colorMask[color]$ 
9:   while cells  $\neq 0$  do
10:    cell  $\leftarrow \text{ctz}(\text{cells})$ ; cells  $\leftarrow \text{cells} \& (\text{cells} - 1)$ 
11:    frontier  $\mid= neighborMask[cell]$ 
12:   end while
13:   return frontier  $\& \sim filledMask$ 
14: end function
15: function DPSOLVE(colorMasks)
16:   if colorMasks in dp then
17:     return dp[colorMasks]
18:   end if
19:   if all cells filled then
20:     return dp[colorMasks]  $\leftarrow 0$ 
21:   end if
22:   best  $\leftarrow \infty$ 
23:   for each color c do
24:     F  $\leftarrow \text{FRONTIERCELLS}(c)$ 
25:     for each cell x in F do
26:       Expand colorMask[c] by x; update filledMask
27:       result  $\leftarrow 1 + \text{DPSOLVE}(\text{colorMasks}')$ 
28:       best  $\leftarrow \min(\text{best}, \text{result})$ 
29:       Undo expansion (backtrack)
30:     end for
31:   end for
32:   return dp[colorMasks]  $\leftarrow \text{best}$ 
33: end function
34: function SOLVE(board)
35:   INITIALIZE(board)
36:   return DPSOLVE(colorMask)
37: end function

```

4 Backtracking

4.1 Description

The algorithm picks one colour and runs DFS to find a path for it, then tries to same for the rest of the colours, if possible. Lets say that the current path for the current picked colour blocks any path for the next colour, then the algorithm undos the move and tries a different path and repeats the same process again.

4.2 Time complexity Analysis

Let the number of cells be N and the number of colors be K

1. The time complexity for finding one path - $\mathcal{O}(N)$
2. Number of possible paths for one colour for board with N cells - $\mathcal{O}(3^N)$
3. Total time complexity - $\mathcal{O}(kN3^N)$

4.3 Feasibility checks

4.3.1 DFS feasibility check

For every move the actual DFS function makes, we run DFS for every unconnected terminal to check if they are still connected and there exists a path between the terminals. This is done in $\mathcal{O}(kN)$, bringing the TC to $\mathcal{O}(k^2N^23^N)$.

4.3.2 Dead region check

For every move the actual DFS function makes, we check if that path leaves any dead region. A dead region is a region of cells that are empty and has no terminals or has only one terminal of a colour, which is done in $\mathcal{O}(N)$, bring the total TC to $\mathcal{O}(kN^23^N)$

Algorithm 5: Solver Backtracking

```
1: function SOLVER(board, index, visited, colors)
2:   if index = |colors| then
3:     return IsCOMPLETEDVISITED(board, visited)
4:   end if
5:   color  $\leftarrow$  colors[index]
6:   (start, end)  $\leftarrow$  FINDTERMINALS(board, color)
7:   if start = (-1, -1) or end = (-1, -1) then
8:     return false
9:   end if
10:  visited[start]  $\leftarrow$  true
11:  paths[color].push(start)
12:  if DFSCOLOR(board, start, end, color, index, visited, colors) then
13:    return true
14:  end if
15:  visited[start]  $\leftarrow$  false
```

```

16:   paths[color].pop()
17:   return false
18: end function
19: function DFSCOLOR(board, current, target, color, colorIndex, visited, colors)
20:   if current = target then
21:     MAKEMOVE(paths[color])
22:     if SOLVER(board, colorIndex+1, visited, colors) then
23:       return true
24:     end if
25:     UNDOMOVE
26:     return false
27:   end if
28:   for each neighbor (nx, ny) of current do
29:     if CANVISIT(board, nx, ny, color, visited) then
30:       visited[nx][ny]  $\leftarrow$  true
31:       paths[color].push(nx, ny)
32:       if DFSFEASIBILITYCHECK(board, visited, colors, colorIndex) then
33:         if DFSCOLOR(board, (nx,ny), target, color, colorIndex, visited, colors)
        then
34:           return true
35:         end if
36:       end if
37:       paths[color].pop()
38:       visited[nx][ny]  $\leftarrow$  false
39:     end if
40:   end for
41:   return false
42: end function

```

4.4 Zig-Zag Variant

Zig-Zag Variant (We couldn't find the actual name for this in the literature) is pretty straight forward. The goal here is to solve the game such that we find path covering all vertices, with an additional challenge to maximize the number of bends your path can take. This allows the existence of problems which has multiple solution in the original problem. Implementing this is as same as before. We maintain a list of all possible solution we get, and find the one which has the highest bends. That's about it. The only cool thing here is probably the fact I wrote it in Zig, which probably made exactly two people excited. Runs about $\mathcal{O}(N3^N)$.

5 Analysis across Algorithms

Algorithm - Radial Weight		
Test Case	Number	Comments
greedy-1	0	This algorithm performs well for boards whose valid paths go around the board
greedy-2	0	
greedy-3	0	
half-split-1	6	Doesn't work well for these because this one simply finds the shortest path
half-split-2	12	

Algorithm - Naïve DnC		
Test Case	Number of Undoes	Comments
greedy-1	0	This algorithm works kinda well
greedy-2	3	
greedy-3	2	
half-split-1	0	Really large puzzle which have straight connections. Need to connect some more terminals
half-split-2	2	

Algorithm - Sorting + BFS/Bellman-Ford		
Test Case	Number of Undoes	Comments
level5	0	Fills the grid
level6	0	Fills the grid
level67	0	No undo operations but fails to fill the grid
level9	1	Bellman-Ford works better than BFS here
level12	2	Requires the bitmasking algorithm due to blocks

Algorithm - Region-Finding Dijkstra		
Test Case	Number of Undoes	Comments
level5	3	Flood fill on the pink pair blocks remaining pairs
level6	6	Red pair blocks other pairs
level67	0	Fills the grid
level9	1	Flood fill proves to be a hindrance here
level12	5	Flood fill proves to be a hindrance yet again

Algorithm - Bellman-Ford + Bitmasking		
Test Case	Number of Undoes	Comments
level5	0	Fills the grid
level6	0	Fills the grid
level67	0	Does not fill the grid due to shortest path limitation
level9	0	Does not fill the grid
level12	0	Checks for all blocks and fully solves the grid

Algorithm - Naive BFS		
Test Case	Number of Undoes	Comments
testcase 1	0	Works well for boards with straight paths
testcase 2	1	
testcase 3	0	
testcase 4	2	Really large spare boards, takes alot more undos because the shortest path is not right
testcase 5	20-24	

Algorithm - Uneven region with DFS		
Test Case	Number of Undoes	Comments
testcase 1	0	Work well for testcases with terminals placed at edges of square
testcase 2	1	
testcase 3	3	
testcase 4	3	Really large spare boards, takes alot more undos because the dfs is always blocking
testcase 5	20-24	

Algorithm - Solver with backtracking		
Test Case	Number of Undoes	Comments
testcase 1	1	Work well for testcases that are dense with terminals
testcase 2	0	
testcase 3	0	
testcase 4	Takes too long	Really large spare boards, too many possibilities without a lot of possibility to prune does possibilities
testcase 5	Takes too long	

Algorithm - DP + Backtracking		
Test Case	Number of Undoes	Comments
Level 1	10	Works well for smaller boards but for larger boards takes long time and requires heuristics to obtain the correct solution.
Level 2	14	
Level 3	4	
Level 4	4	
Level 5	4	

6 Gallery

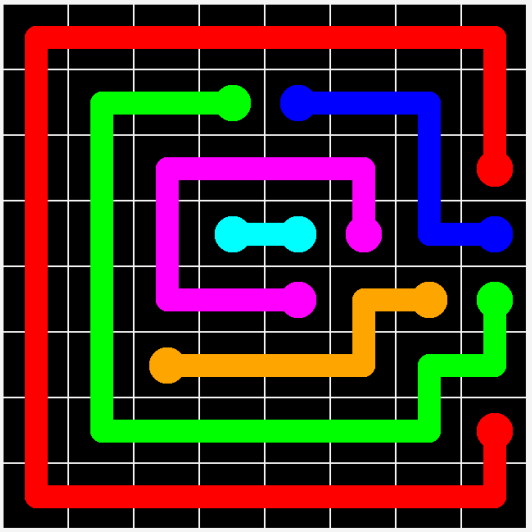


Figure 1: Greedy Weight Test Case

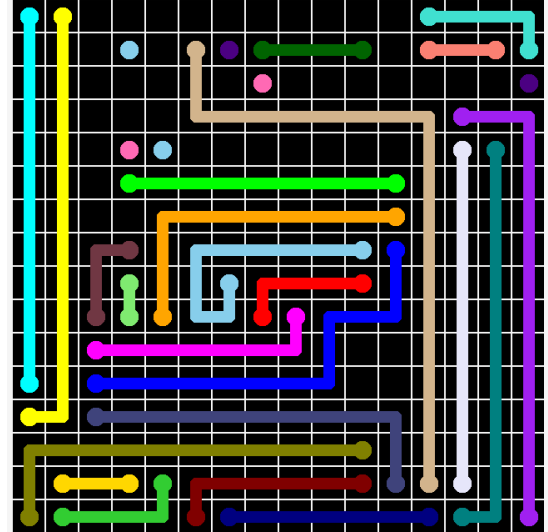


Figure 2: Naive DnC Test Case

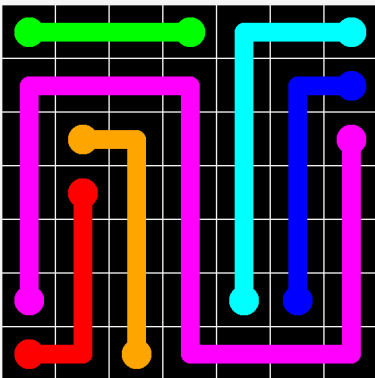


Figure 3: level5

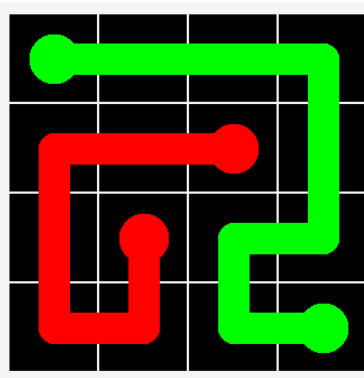


Figure 4: level67

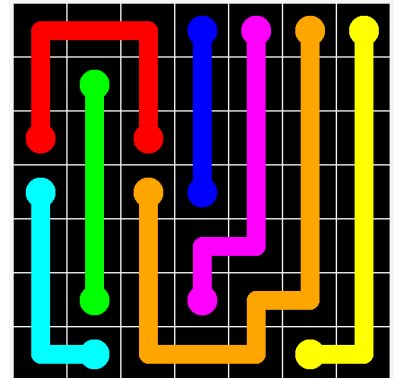


Figure 5: level12

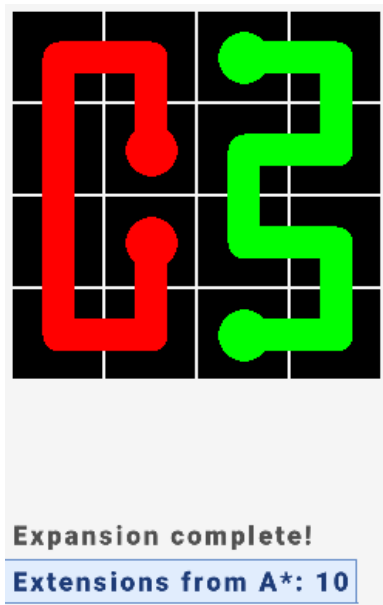


Figure 6: level1

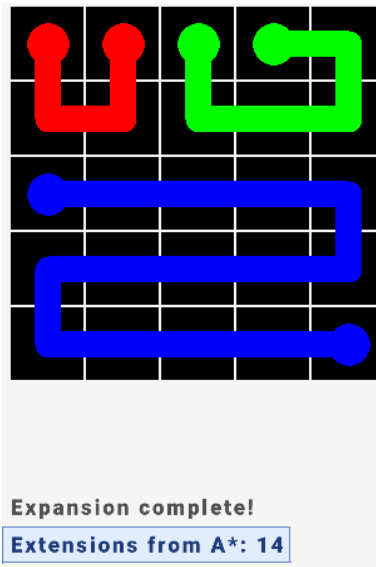


Figure 7: level2

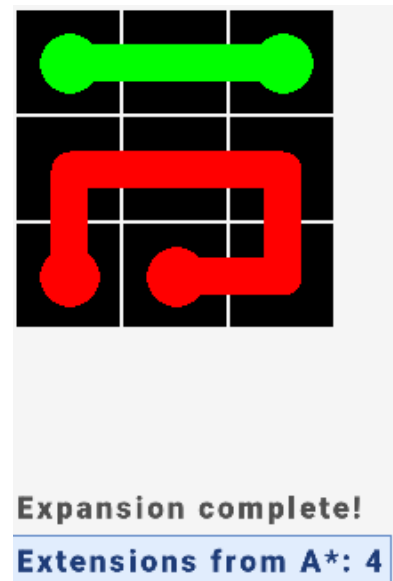


Figure 8: level3

7 Conclusion and Future Directions

Now that we have reached the end of this (and the semester), I think this is the right time to contemplate what we have achieved. We started by choosing this game from all the other cool games. We tried to work our way around simple traversals, clever weighting techniques, heuristics, and finally a standard solution. Now what did it all amount to? What peak have we conquered? Was it even about conquering a peak?

This exercise wasn't just a playground to experiment and fiddle around with some funny looking puzzle. It embodies the very essence of living. Flow Free is how the problems we face in real life are. You try to come up with some strategy which looks like an easy solution to your problem, but life has it's own kick. So you try something new, it gives you something more back. Life is in NP-hard, but living is in P. Would you have enough time to go through all possible case, and figure out which decisions in life will lead to the minimal suffering? Idealism is expensive. But do we stop? No. We persist. And that's what we did this semester.

We don't know what the future will offer us. Heck, we don't know what will happen tomorrow. Life has it's own little great thing. Let us look forward to it, and become the greatest we can.



8 LLM Disclosure

Here is a series of prompts (or the kind of prompts) given to LLMs, to help us assist during the course of this project.

1. Makefile fixes (with code snippets)
This prompt was given to generate the correct makefile which aligns with the user's C++ compiler and Raylib location.
2. UI offset fixes (with code snippet)
This prompt was given to fix an alignment issue with the game window and the grid. It resolved the issue where the padding and grid offset parameters were used interchangeably.
3. LaTeX pseudocode formatting (pseudocode given)
This prompt was given to convert the pseudocode entered by the user to make it LaTeX-style using the `algpseudocode` and `algorithmic` packages.
4. If a color does not get a solution path inside its segmented region, what should be done? should we expand the region or discard the segmentation?
5. What if connecting one color first blocks another color that had a valid path earlier?
6. If two colors have the same number of candidate paths, how do we decide which one to connect first?
7. How to find perpendicular points to a line?
8. How to check if a point is inside a polygon?
9. Game window fixes
Adding buttons for all the solvers messed up the game window UI along with the required offset and padding fixes.
10. Added debug flags for bitmask traceback error handling